BITRASER FILE ERASER - INTERVIEW PREPARATION GUIDE

Comprehensive Technical Q&A;, Architecture Deep-Dive & System Programming Guide for C++ / Qt Developers

Project: BitRaser File Eraser (Stellar Data Recovery)	**Target Role:** C++ / Qt / Windows Systems Engineer
Key Technologies: C++, Qt 5/6, Win32 Direct Disk I/O, MariaDB, REST API, Windows Scheduler	**Domain:** Data Sanitization, Forensic Security & Compliance

SECTION 1: PROJECT OVERVIEW & CANDIDATE ELEVATOR PITCH

Q1: Can you introduce this project? What is BitRaser File Eraser, and what problem does it solve? [OVERVIEW]

Elevator Pitch:

- **BitRaser File Eraser** is an enterprise-grade and consumer data sanitization software developed in **C++ and Qt** for Microsoft Windows. Its primary purpose is the **permanent, forensically unrecoverable erasure** of confidential files, folders, system/internet traces, volume slack, and free space.
- **The Problem it Solves:** When users delete files using standard OS operations (Shift+Delete or Recycle Bin), the actual binary data is *never erased*—only the pointers in file system allocation tables (e.g., NTFS Master File Table, FAT Directory Entries) are marked as free. Anyone using commercial or free data recovery software can retrieve sensitive data. BitRaser complies with **22+ international wiping standards** (such as **NIST 800-88, DoD 5220.22-M, Peter Gutmann**) to overwrite raw sectors, guaranteeing compliance with **GDPR, HIPAA, PCI-DSS, and ISO 27001**.
- **Key Architectural Modules:**
- **1. Core Wiping Engine (WipingAlgo / DiskIO):** Low-level Win32 sector-by-sector overwriting with buffer alignment and verification passes.
- **2. Traces Erasure Engine:** Deep scrubbing of OS artifacts (Prefetch, Temp, Event logs, MRU) and browser databases (Chrome, Firefox, Edge).
- **3. Automation & Scheduling:** Native integration with Windows Task Scheduler 2.0 COM API and Windows Explorer context menus.
- **4. Auditing & Reporting:** Digitally signed, tamper-evident PDF/XML Certificates of Erasure with hardware serial numbers and timestamps.

Q2: What was your specific role and key technical contributions in this codebase? [ROLE & TECH]

- **Core Erasure Engine Development:** Implemented and fine-tuned multi-pass wiping algorithms (DoD 3/7 pass, NIST Clear, Gutmann) and low-level sector read/write routines using Win32 API (CreateFileW, WriteFile, DeviceIoControl).
- **Asynchronous Processing & Threading:** Decoupled heavy, blocking disk I/O operations from the Qt main GUI thread using worker threads (QThread/BTask) and throttled signal-slot emissions to maintain responsive 60 FPS UI rendering.
- **Trace Sanitization:** Authored deep cleaning routines for internet traces (SQLite history, cookies, cache) and Windows OS artifacts (Prefetch, JumpLists, Recycle Bin raw clusters).
- **Cloud & Licensing Integration:** Integrated libcurl for RESTful communication with the BitRaser Cloud Console, encrypting JSON payloads using AES-128-CBC and hardware machine fingerprinting.
- **Audit Report Generation:** Integrated Debenu PDF Library to dynamically generate certified, tamper-evident PDF Certificates of Erasure with technician digital signatures and custom logos.

SECTION 2: DATA SANITIZATION STANDARDS & ALGORITHMS

Q3: Which data erasure standards are implemented in BitRaser, and how do they work? [ALGORITHMS]

The codebase implements **22 distinct erasure standards** (defined in WipingWrapper.h and WipingAlgo.cpp):

- **Zeroes (1 Pass):** Overwrites all sectors with 0x00. Quickest sanitization.
- **Random (1 Pass):** Overwrites every sector with cryptographically generated pseudo-random bytes seeded by system time.
- **DoD 5220.22-M (3 Passes):** Pass 1: Fixed character (0x00) + Verify; Pass 2: Complement (~0x00 = 0xFF) + Verify; Pass 3: Pseudo-random bytes + Verify.
- **DoD 5220.22-M ECE (7 Passes):** Executes DoD-3 pass, followed by an additional random pass, then 0xFF, its complement, and final random data.
- **NIST 800-88 Rev 1 (Clear & Purge):** Industry gold standard. *Clear* overwrites user-addressable storage space with a single pass plus read verification. *Purge* utilizes internal hardware commands (ATA Secure Erase / Sanitize) to purge non-addressable spare sectors and reallocated blocks.
- **Peter Gutmann (35 Passes):** 4 pseudo-random passes, 27 specific magnetic flux transition patterns designed for older MFM/RLL encoding schemes, and 4 final random passes.
- **Other Standards:** US Army AR380-19 (3 passes), US Air Force AFSSI-5020 (3 passes), Russian GOST R 50739-95 (2 passes), German VSITR (7 passes), British HMG IS5 Baseline & Enhanced, NATO, and RCMP TSSIT OPS-II.

Q4: Why do we use complement patterns (~0x00) and random data instead of just writing zeros? [DEEP DIVE]

- **Physics of Magnetic Storage:** On magnetic hard disk drives (HDDs), bits are stored by aligning microscopic magnetic domains. Writing only zeros leaves slight magnetic hysteresis traces along the edges of the physical track, which theoretically could be reconstructed using specialized laboratory equipment like Magnetic Force Microscopy (MFM).
- **Complement Inversion:** Writing a pattern (e.g. 0x55 = 01010101) immediately followed by its bitwise complement (0xAA = 10101010) forces every single magnetic domain to flip polarity, neutralizing residual hysteresis.
- **Modern Drives vs Legacy:** While modern high-density PRML/SMR hard drives make recovering data after a single verified zero pass nearly impossible, government and military compliance certifications (DoD, NIST) strictly require multi-pass complement and pseudo-random patterns before equipment can be decommissioned.

Q5: How does SSD erasure fundamentally differ from HDD erasure? Why is software overwriting tricky on SSDs? [SSD VS HDD]

- **Flash Translation Layer (FTL) Abstraction:** On SSDs, Logical Block Addresses (LBAs) do NOT correspond to fixed physical flash cells. The FTL constantly redirects writes to different physical NAND blocks to achieve **Wear Leveling**.
- **Over-Provisioning & Inaccessible Blocks:** SSDs have spare NAND capacity (over-provisioning) and retired bad blocks that the OS cannot directly address via standard LBA sector writes. A software overwrite to LBA 500 writes to a new NAND cell, leaving the stale cell untouched until garbage collection runs.
- **How BitRaser Handles SSDs:**
- 1. *Free Space Wipe + TRIM:* Overwrites unused clusters and invokes the Windows TRIM API (FSCTL_FILE_LEVEL_TRIM), informing the SSD controller that those physical blocks contain invalid data for immediate background erase.
- 2. *Firmware / ATA Secure Erase:* For full volume/disk sanitization, BitRaser invokes ATA Secure Erase / NVMe Format commands directly via IOCTL, which instructs the hardware controller to apply a voltage spike across all flash blocks simultaneously, sanitizing the entire drive including over-provisioned areas.

Q6: Explain how Overwrite Verification works. What is the difference between Quick and Deep verification? [VERIFICATION]

- **Verification Logic (DiskIO.cpp -> Verification()):** After writing the pattern buffer to disk, the application immediately reads the sectors back into memory and compares them byte-for-byte against the expected pattern buffer.
- **Quick Verification (QUICKVERIFICATION):** Validates a statistical sample of sectors (e.g., 1 sector block out of every 4, or sampling boundaries of clusters). This provides high confidence while minimizing verification time on multi-terabyte drives.
- **Deep Verification (DEEPVERIFICATION):** Performs a 100% complete read-back of every single wiped sector across the entire disk or file. If a single byte differs or an I/O read error occurs, the process logs a verification failure and flags the audit report.

SECTION 3: LOW-LEVEL WIN32 PROGRAMMING & FILE SYSTEM INTERNALS

Q7: How does BitRaser open physical disks and volumes? Which Win32 APIs and flags are mandatory? [WIN32 API]

- **Device Handle Acquisition:**

```
HANDLE hDevice = CreateFileW(lpbDiskString, GENERIC_READ | GENERIC_WRITE, FILE_SHARE_READ |
FILE_SHARE_WRITE | FILE_SHARE_DELETE, NULL, OPEN_EXISTING, FILE_FLAG_NO_BUFFERING |
FILE_FLAG_WRITE_THROUGH, NULL);
```

- **Physical vs Logical Path Formats:**
- • Physical Disk: \\.\PHYSICALDRIVE0 (accesses entire physical disk, bypassing partition tables).
- • Logical Volume: \\.\C: (accesses volume clusters directly).
- **Crucial Flags:**
- • FILE_FLAG_NO_BUFFERING: Completely bypasses Windows system cache buffers. Data is written directly from application memory to the storage media. *Requirement:* Memory buffers and byte counts must be integer multiples of the disk sector size (512 or 4096 bytes).
- • FILE_FLAG_WRITE_THROUGH: Prevents intermediate caching; ensures WriteFile does not return until the hardware controller commits the data to non-volatile storage.
- **64-Bit Seeking:** Uses SetFilePointer / SetFilePointerEx with a 64-bit LARGE_INTEGER offset calculated as: SectorNo * SECTSIZE.

Q8: What is Slack Space, and how does BitRaser wipe it without corrupting valid data? [FILE SYSTEMS]

- **Understanding Slack Space:** File systems allocate space in cluster increments (e.g., 4 KB = 4096 bytes). If a file's logical size is 1500 bytes, the file system allocates an entire 4096-byte cluster.
- • *RAM Slack:* The remaining bytes from the end of the file (byte 1500) to the end of the 512-byte sector (byte 1536). Historically padded with RAM contents by old OS kernels.
- • *Cluster Slack:* The unused space from the end of that sector (byte 1536) to the end of the allocated cluster (byte 4096). This area frequently holds fragments of previously deleted confidential files!
- **How BitRaser Erases Slack Space:**
- 1. Queries the file's physical allocation clusters using Win32 API FSCTL_GET_RETRIEVAL_POINTERS.
- 2. Calculates the exact offset between logical End-Of-File (EOF) and physical cluster boundary.
- 3. Overwrites only the residual bytes from EOF to the end of the allocated cluster with wiping patterns, preserving the original file content intact.

Q9: How does NTFS store files, and what happens to the Master File Table (MFT) records after file erasure? [NTFS INTERNALS]

- **NTFS Architecture:** In NTFS, every file and folder has a 1024-byte record inside the Master File Table ($MFT). Small files (< 700 bytes) are stored 'resident' directly inside the MFT record itself!
- **The Forensic Leak:** If you only overwrite the external data clusters and delete the file, the MFT record still retains the file name, file creation/modification timestamps, file size, permissions, and directory tree path!
- **BitRaser's MFT & Journal Scrubbing:**
- 1. Overwrites the file's data clusters with the selected algorithm.
- 2. Truncates and overwrites the resident attributes inside the MFT entry.
- 3. Cleans metadata from NTFS transaction journals: $LogFile and $UsnJrnl (Update Sequence Number Journal), preventing forensic software from reading historic file transactions.

Q10: How does the software handle files that are locked by Windows or other processes (ERROR_SHARING_VIOLATION)? [LOCKED FILES]

- **The Challenge:** When an active process or system service holds an exclusive handle on a file (e.g. system logs, antivirus DBs), CreateFile fails with error code 32 (ERROR_SHARING_VIOLATION).
- **BitRaser's Multi-Tier Resolution:**
- • **Tier 1 - Process Identification:** Uses Windows Restart Manager APIs (RmStartSession, RmRegisterResources, RmGetList) to identify which process holds the lock and prompts the user to close it gracefully.
- • **Tier 2 - Non-Exclusive Share Flags:** Attempts opening with FILE_SHARE_READ | FILE_SHARE_WRITE | FILE_SHARE_DELETE.
- • **Tier 3 - Boot-Time Scheduling:** For core OS files, BitRaser registers the file for deletion during the next Windows startup sequence before file locking services start, leveraging MoveFileEx(..., MOVEFILE_DELAY_UNTIL_REBOOT) and custom boot-time drivers.

Q11: How does BitRaser handle file paths exceeding the standard 260-character MAX_PATH limit? [LONG PATHS]

- Standard Windows APIs enforce a legacy limit of 260 characters (MAX_PATH). Deep folder hierarchies easily exceed this.
- In bitraserfileeraser.h, the macro UNC_PATH_PREFIX is defined as L"\\?\".
- By prepending \\?\ to absolute file paths (e.g. \\?\C:\Deep\Folder\...\file.txt), the Win32 API bypasses path parsing and supports paths up to **32,767 Unicode characters**.
- All file operations throughout the codebase utilize wide-character Win32 APIs (CreateFileW, FindFirstFileW, DeleteFileW).

SECTION 4: SYSTEM, BROWSER & APPLICATION TRACES ERASURE

Q12: What are 'System Traces' and how does BitRaser sanitize them? [TRACE CLEANING]

System traces are digital footprints left by Windows during daily operation (implemented in systemtraces.cpp):

- **Windows Prefetch (C:\Windows\Prefetch*.pf):** Records binary paths and launch frequency of every executed application. Overwritten and cleared.
- **Recycle Bin ($Recycle.Bin):** Overwrites the raw clusters of deleted files and purges index metadata ($I... and $R... files).
- **Recent Documents & Jump Lists:** Cleans LNK shortcuts and binary jump lists in %APPDATA%\Microsoft\Windows\Recent.
- **Windows Event Logs (*.evtx):** Cleans security and application event logs that record logon sessions and process executions.
- **Temporary Files (%TEMP%, C:\Windows\Temp):** Overwrites and deletes orphaned cache and installer artifacts.
- **Registry MRU Lists:** Sanitizes Most Recently Used (MRU) lists stored in the Windows Registry (e.g. Run dialog history, TypedURLs, Open/Save file dialog history).

Q13: How does the software sanitize browser traces (Chrome, Firefox, Edge) when SQLite databases are locked? [BROWSER TRACES]

- **Browser Data Storage:** Modern browsers store browsing history, cookies, download history, and session tokens inside SQLite database files (e.g. History, Cookies, Web Data) under %LOCALAPPDATA%.
- **Handling SQLite WAL Locks:**
- 1. Detects if browser processes (chrome.exe, firefox.exe, msedge.exe) are running using CreateToolhelp32Snapshot and prompts the user to terminate them.
- 2. If browsers are closed, BitRaser connects via SQLite or directly overwrites and truncates the database files, including their Write-Ahead Log (-wal) and Shared Memory (-shm) files.
- 3. Overwrites browser cache directories containing cached web images, scripts, and temporary session files.

SECTION 5: C++, QT & SOFTWARE ARCHITECTURE

Q14: What design patterns are used in this project? Give examples from the code. [DESIGN PATTERNS]

- **Singleton Pattern:** Implemented in BitRaserFileEraser::GetInstance(). Provides a single globally accessible instance managing application lifecycle, active settings, database connection, and licensing state.
- **Wizard Pattern:** Used in WipingWizard and ScheduleWipingWizard. Manages complex multi-step user workflows (Selecting targets -> Configuring wiping algorithm & verification -> Pre-wipe summary -> Execution & real-time progress -> Post-wipe report).
- **Command / Worker Pattern:** Encapsulates scheduled and ad-hoc background wiping actions inside BTask and btaskbase classes.
- **Observer Pattern (Signals & Slots):** Qt's Signal and Slot mechanism decouples the low-level I/O wiping worker threads from GUI updates (e.g. progress bar percentages, sector speed, time remaining, file tree updates).

Q15: How do you prevent UI freezing during intensive raw disk I/O? How is thread communication handled? [THREADING & GUI]

- **The Bottleneck:** Disk wiping executes millions of synchronous WriteFile and ReadFile calls. Running this on the Qt GUI main thread blocks the Windows message pump, causing the window to freeze and trigger Windows 'Not Responding' status.
- **Multi-Threaded Architecture:**
- 1. The main Qt GUI thread handles user interaction, rendering, and window events.
- 2. Erasure runs in a dedicated background worker thread (QThread).
- 3. Communication uses Qt Queued Connections (Qt::QueuedConnection) across thread boundaries.
- **Event Queue Throttling:** Emitting a Qt signal on *every single 512-byte sector* would flood the event queue and choke the UI thread. Instead, progress signals are throttled to emit only after processing large chunks (e.g. every 16 MB or every 250 milliseconds).

Q16: How is Single Instance Application enforcement implemented in main.cpp? [SINGLE INSTANCE]

- In main.cpp, the application uses QSharedMemory with a unique GUID:

```
QSharedMemory shared("32s60669-ab94-4a84-88bc-b964790f7g02"); if (!shared.create(512, QSharedMemory::ReadWrite)) { // Second instance detected - display warning dialog and terminate MassageBoxDlg msg("Warning", "An instance is already running.", ...); msg.exec(); return 0; }
```

- If shared.create() returns false, the operating system already holds this shared memory segment from an active instance. The duplicate instance alerts the user and exits immediately.

Q17: How are Themes (Dark/Light) and Internationalization (i18n) handled? [UI FEATURES]

- **Theming (BRF_Theme):** Managed via registry key. The application dynamically applies custom Qt Style Sheets (QSS) to re-skin dialogs, table widgets, buttons, and switches between light/dark asset icon sets (e.g. IDI_WIPE_NOW vs IDI_WIPE_NOW_B).
- **Multi-Language i18n (m_Language):** Supports 7 languages: English, German, French, Japanese, Spanish, Italian, and Dutch. Implemented using QTranslator and compiled binary translation files (.qm) loaded from Qt resource files (Resource.qrc). Date formatting is localized using Win32 API SetLocaleInfoW.

SECTION 6: CLOUD INTEGRATION, LICENSING & CRYPTOGRAPHY

Q18: How does the software generate a Machine Fingerprint (Digital ID) and enforce licensing? [LICENSING]

- **Machine Fingerprinting (GetDigitalID() in bitraserlicenseservice.cpp):**
- • Extracts immutable hardware attributes: Motherboard UUID, CPU ID / Serial, Primary Network Adapter MAC Address, and System Drive Volume Serial Number.
- • Hashes these identifiers together into a unique, tamper-proof Digital Machine ID.
- **Online Activation:** Posts the machine ID and user's activation key to Stellar's web licensing server via HTTPS (fncASActivateLicense). If valid, returns an encrypted license string storing allowable erasure counts and expiry.
- **Anti-Tamper & Anti-Cloning:** If an activated image is cloned onto another machine or VM, the Digital ID mismatch invalidates the local license cache, reverting the software to Demo mode.

Q19: How is data secured during transit to the BitRaser Cloud Console? [CLOUD SECURITY]

- **Transport Security:** Built on top of libcurl utilizing TLS 1.2/1.3 with a bundled trusted root certificate authority bundle (cacert.pem).
- **Application Payload Encryption:**
- • JSON data packets are encrypted using **AES-128-CBC** (AES_128_CBC_encrypt) with randomized Initialization Vectors (IV) and PKCS7 padding before dispatch.
- • Ciphertext is encoded with **Base64** (Base64Encode) for safe JSON HTTP transmission.
- • Local sensitive credentials stored in the Windows Registry are obfuscated using a custom salted XOR algorithm (encryptPassToXOR).

Q20: How are PDF Certificates of Erasure generated, and how is audit compliance verified? [PDF CERTIFICATES]

- **PDF Engine (reportsaving.cpp):** Utilizes the Debenu PDF Library (Quick PDF) to generate digitally verifiable, tamper-evident Certificates of Erasure.
- **Forensic Audit Content:** Each certificate documents:
- • Disk/Device Details: Model, Serial Number, Disk Capacity, Bus Type.
- • Erasure Metadata: Algorithm used, Number of Passes, Verification Result (100% Passed), Total Bytes Wiped, Duration.
- • Legal Audit Trail: Technician Name, Validator Name, Department, Organization Logo, Watermark, and Timestamps.
- **Tamper Verification:** A cryptographic SHA-256 hash of the process data is generated and synchronized with the BitRaser Cloud Console. If someone edits the generated PDF file, the certificate hash will fail validation against the cloud audit log.

SECTION 7: TASK SCHEDULING, SHELL INTEGRATION & DATABASE

Q21: How does the automated Task Scheduler work without launching the main user interface? [TASK SCHEDULING]

- **Windows Task Scheduler 2.0 COM API:** Implemented in btask.cpp using taskschd.h (ITaskService, ITaskFolder, ITaskDefinition, ITriggerCollection).
- **Supported Triggers:** Daily, Weekly, Monthly, At System Logon, and At System Shutdown.
- **Unattended Execution Flow:**
- 1. Windows Task Scheduler executes brf.exe with a specific command-line argument starting with $BRFST- (e.g. $BRFST-EraseTraces).
- 2. In main.cpp, the flag launchedByScheduler is set to true.
- 3. Instead of launching the full interactive main window, it instantiates ScheduleWipingWizard in a silent/minimized mode, runs the assigned wipe job, logs results into LocalDb, and terminates cleanly.

Q22: How does the Windows Explorer right-click context menu erasure work? [EXPLORER SHELL]

- **Shell Verb Registration:** During installation, BitRaser registers Windows Shell context verbs under HKCR*\shell\BitRaser and HKCR\Directory\shell\BitRaser.
- **Execution Mechanism:**
- 1. When a user right-clicks selected files in Explorer and clicks 'Erase with BitRaser', Windows launches brf.exe with parameter '2' or writes the target paths into a temporary control file BitRaser_Ext.txt in %LOCALAPPDATA%.
- 2. main.cpp calls isRightClickOrCommandLine() to parse the file paths, validates their existence and permissions, and launches the wiping wizard targeted specifically at those paths without requiring the user to navigate the entire app.

Q23: What local database is used, and what data structures are stored? [LOCAL DATABASE]

- **Database Engine (LocalDb.cpp):** Embedded MariaDB / MySQL client library (libmariadb.dll).
- **Key Data Tables & Structures:**
- • FileDataInfo: Stores historical logs of every erased file: Name, Absolute Path, Status (Success/Failed), Start/End timestamps, File size, and ProcessID.
- • DeletedDataInfo: Stores free space erasure logs: Drive letter, total unused space, erased space, duration, and status.
- • ProcessDataInfo: High-level process summary: Algorithm used, total files wiped, verification status, and error codes.
- • UserDataInfo: Stores auditor details: Technician name, validator name, department, custom headers, and signature file paths.

SECTION 8: TOUGH INTERVIEW SCENARIOS, DEBUGGING & BEHAVIORAL

Q24: Describe a difficult bug you encountered in this project and how you solved it. [TROUBLESHOOTING]

Top Technical Bug: The 4K Native (4Kn) Unbuffered Write Error (Error 87)

- **Symptom:** On newly released external NVMe drives and 4Kn enterprise HDDs, disk wiping failed immediately with Win32 Error 87 (ERROR_INVALID_PARAMETER), while the exact same code worked fine on older 512-byte sector drives.
- **Root Cause Analysis:** When using FILE_FLAG_NO_BUFFERING in CreateFile, Windows strictly enforces that both the memory buffer memory address (pointer) AND the number of bytes written must be an exact integer multiple of the disk's physical sector size! The legacy buffer size was hardcoded to 512 bytes.
- **Solution:**
- 1. Implemented a dynamic query using DeviceIoControl(hDevice, IOCTL_DISK_GET_DRIVE_GEOMETRY_EX, ...) to retrieve the exact BytesPerSector (e.g. 4096 bytes).
- 2. Replaced heap memory allocations (malloc/new) with VirtualAlloc, which guarantees that memory buffers are aligned to OS page boundaries (4096-byte boundary).
- 3. Adjusted the wiping chunk loop to write in exact multiples of 4096 bytes. The bug was completely resolved across all drive formats.

Q25: What happens if power is cut or the system crashes midway through a multi-pass wipe? [EDGE CASE]

- **Data Recoverability:** Sectors that have already been overwritten with random/zero patterns are permanently destroyed and completely unrecoverable. Sectors that were not yet reached retain their original state.
- **System State & Integrity:**
- • If wiping an entire volume/drive, the partition table and file system superblock may already be destroyed. Windows will prompt that the drive needs formatting.
- • BitRaser tracks state incrementally in LocalDb. If restarted after an unexpected crash, the process status is marked as 'Aborted / Incomplete'.
- • Under no circumstances will a Certificate of Erasure be generated for an interrupted wipe, strictly preventing false audit compliance.

Q26: How do you prove to an auditor that data erased by BitRaser is truly unrecoverable? [COMPLIANCE]

- **1. Internal Bit-Level Read Verification:** Every sector can undergo 100% read-back verification against the expected algorithm pattern.
- **2. Forensic Carving Validation:** The wiped media is scanned using commercial recovery tools (Stellar Data Recovery, EnCase, FTK Imager, Autopsy). Technicians verify that zero file signatures (e.g. JPEG, PDF, DOCX magic bytes) can be salvaged.
- **3. Hex Examination:** Low-level hex editors (WinHex, HxD) verify that physical LBA sectors contain uniform zeros or random entropy without any readable text or file table remnants.
- **4. International Lab Certifications:** BitRaser software is certified by independent bodies such as ADISA (Asset Disposal & Information Security Alliance) and compliant with NIST 800-88 standards.

SECTION 9: CANDIDATE REVISION CHEAT SHEET

Topic	Key Functions / Concepts to Mention in Interview
Win32 Disk APIs	CreateFileW, WriteFile, ReadFile, SetFilePointerEx, DeviceIoControl, VirtualAlloc, FILE_FLAG_NO_BUFFERING, FILE_FLAG_WRITE_THROUGH
Drive Path Syntax	Physical Drive: \\.\PHYSICALDRIVE0 | Logical Volume: \\.\C: | Extended Path: \\?\C:\LongPath...
Key Algorithms	NIST 800-88 Clear (1-pass + verify), DoD 5220.22-M (3-pass: zero, complement, random), Gutmann (35 passes), HMG IS5, VSITR
File System Internals	Cluster Slack, RAM Slack, Master File Table (MFT) record overwrite, $LogFile & $UsnJrnl wiping, FTL & TRIM on SSDs
Qt Architecture	QThread (worker threads), Queued Connection signals/slots, QSharedMemory (single instance), QWizard, QTranslator (i18n)
Cloud & Security	libcurl over TLS 1.3, AES-128-CBC payload encryption, Base64 encoding, Machine Fingerprinting (Digital ID), Debenu PDF Library
Task Scheduler	Windows Task Scheduler 2.0 COM API (taskschd.h, ITaskService), $BRFST- command-line flags, Explorer context menu verbs